This test does not depend in any way on the actual HTTP request; instead it defines the API of the adapter. The test creates a User instance and then creates a fake adapter using RSpec's instance_double method to ensure that any method you stub actually exists in the AvatarAdapter class. Then, spies are used to set the expectation that the avatar will receive the image_url method.

> **Prescription 38** Use the adapter to test client behavior without being dependent on the server API.

The "when" part of this test is the last line: the call to the user.avatar_url method. At this point, you've set up the following expectations for what happens next:

- The AvatarAdapter class will be sent the new method with the user instance as an argument.
- The resulting instance of AvatarAdapter will be sent the image_url method.

To make this test pass, you need a method in User:

external/01/app/models/user.rb

```
def avatar_url
  adapter = AvatarAdapter.new(self)
  adapter.image_url
end
```

And you need a new AvatarAdapter class:

external/01/app/models/avatar_adapter.rb

```
class AvatarAdapter
  def initialize(user); end

  def image_url; end
end
```

Right now the AvatarAdapter class is just a skeleton, which is all you need to make the unit tests pass.

Now it's time to get that adapter done.

Why an Adapter?

Using an adapter class to mediate interaction with the external service is a good idea even when, like Twitter, the external service already has a Ruby gem. The adapter encapsulates logic that is specific to the interaction between your application and the service.